[Trung Nguyen] Boot Process (ARM), U-Boot, Linux Kernel, Root Filesystem & Buildroot

1. U-Boot Bootloader

Das U-Boot (often shortened to u-boot) is an open-source bootloader widely used in small embedded systems. It has built-in support for many architectures, including 68k, ARM, Blackfin, MicroBlaze, MIPS, Nios, SuperH, PPC, RISC-V and x86.

Its main job is to initialize the hardware, load the other OS components (Linux kernel, rootfs, device tree) into RAM, and then hand control over to the Linux kernel.

U-Boot's strengths come from the design principles its developers set out:

- Keep it Small
- Keep it Fast
- Keep it Simple
- Keep it Configurable
- Keep it Debuggable
- Keep it Usable
- Keep it Maintainable
- Keep it Beautiful
- Keep it Open

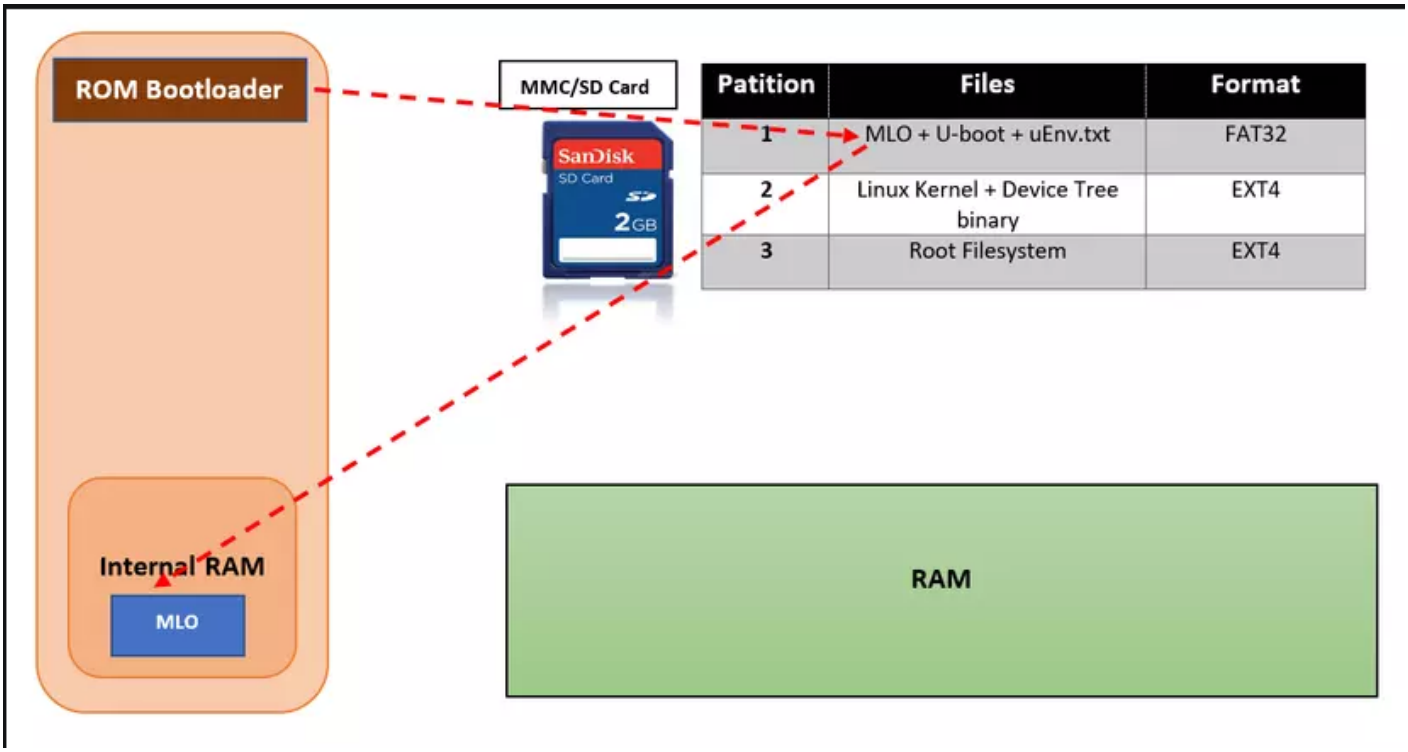
SoC ROM Bootloader

When the system powers on for the first time, or is reset, control belongs to the reset vector: a piece of assembly code written in advance by the chip manufacturer. The reset vector points to the memory address holding the first boot code, namely the boot ROM. Without a reset vector, the processor would not know where to start executing.

The main job of the boot ROM is to copy the contents of the "MLO" file, also called the Second Program Loader (SPL), into internal RAM (IRAM) and execute it.

Because boot ROM memory is quite small, the ROM code is limited to initializing only the hardware needed to load the SPL, such as MMC/eMMC, SD card and NAND flash. This hardware is collectively called the boot device.

Example: loading from an SD card:

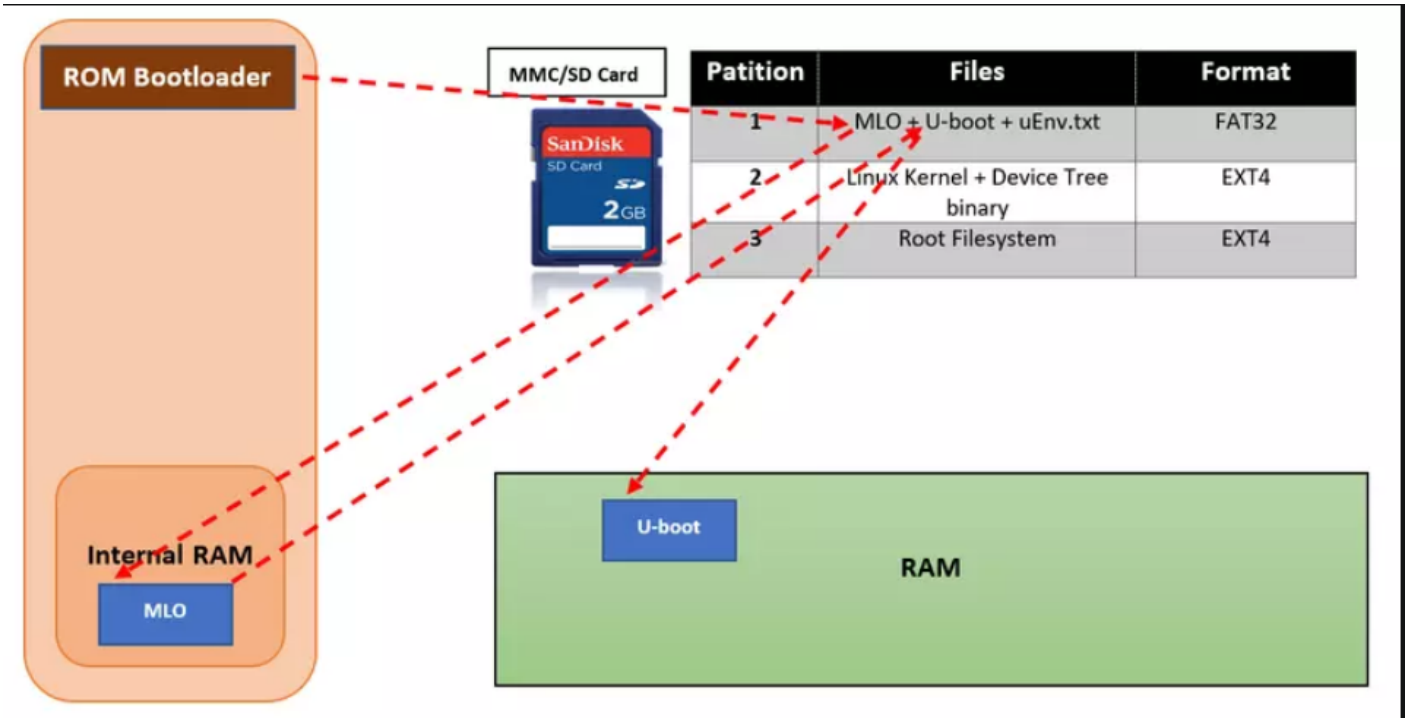


Which boot device the ROM code chooses (SD card, flash, etc.) depends on how the boot pins are configured through switches/jumpers on the hardware.

Second Program Loader (SPL)

The SPL is a secondary loader. Its main job is to set up further required components such as the DRAM controller, eMMC, etc., and then load U-Boot to address `CONFIG_SYS_TEXT_BASE` in RAM.

In short, the purpose of the SPL is to get U-Boot into RAM.

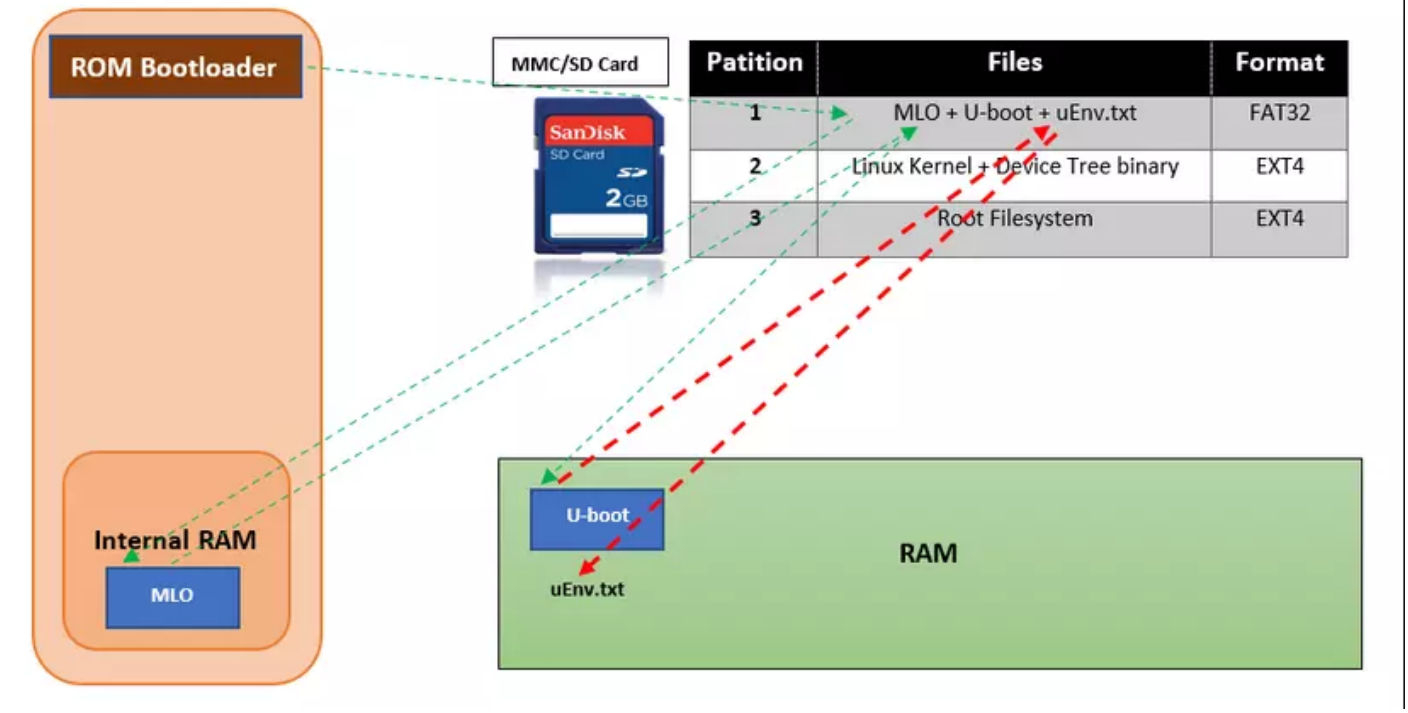


U-Boot

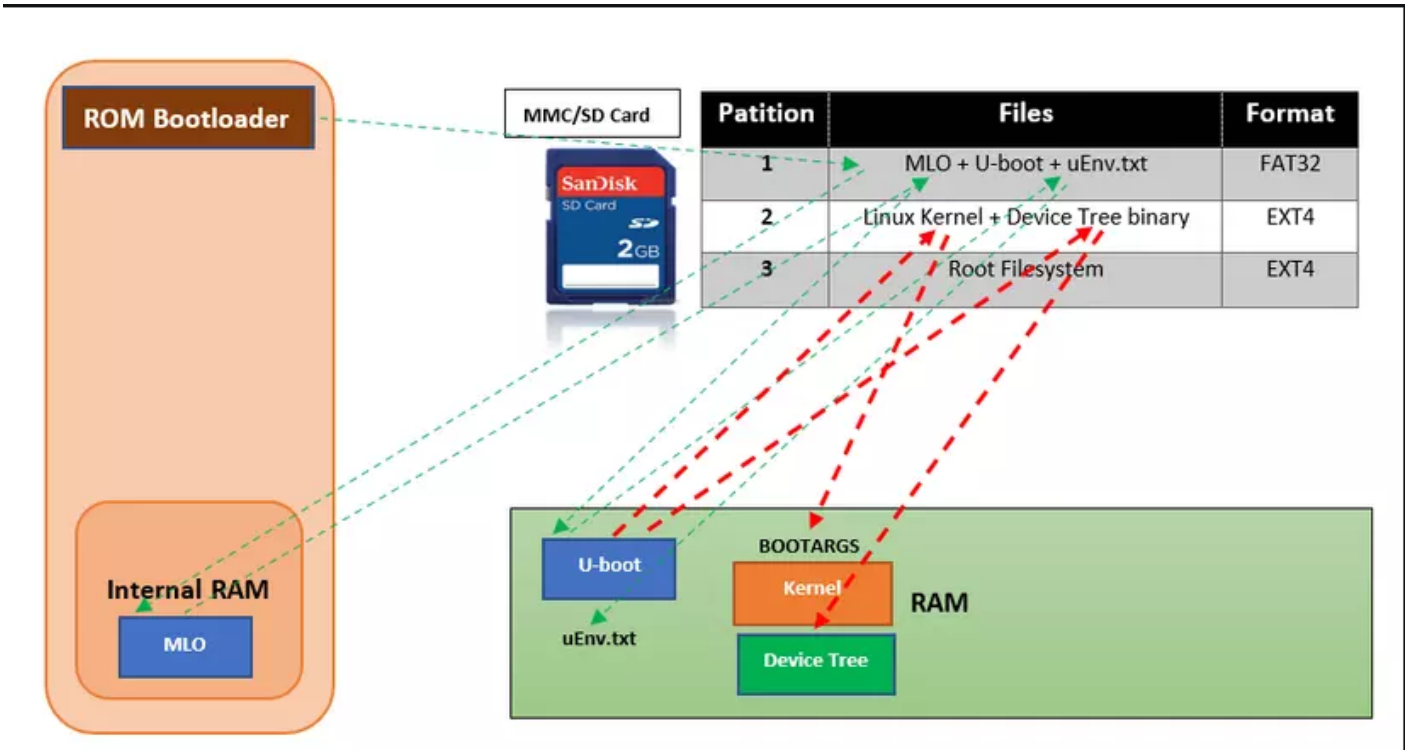
After being loaded into RAM, U-Boot performs relocation: it moves itself to the `relocaddr` address in RAM (usually near the end of RAM) and jumps to the relocated U-Boot code.

U-Boot then checks whether a `uEnv.txt` file exists. If it does, U-Boot loads it into RAM in the next step.

`uEnv.txt` is a boot script that defines configuration parameters and kernel parameters. These parameters already have defaults configured in U-Boot, but we can add, modify or remove them through `uEnv.txt`. Loading `uEnv.txt` is optional, meaning the file may or may not be present.

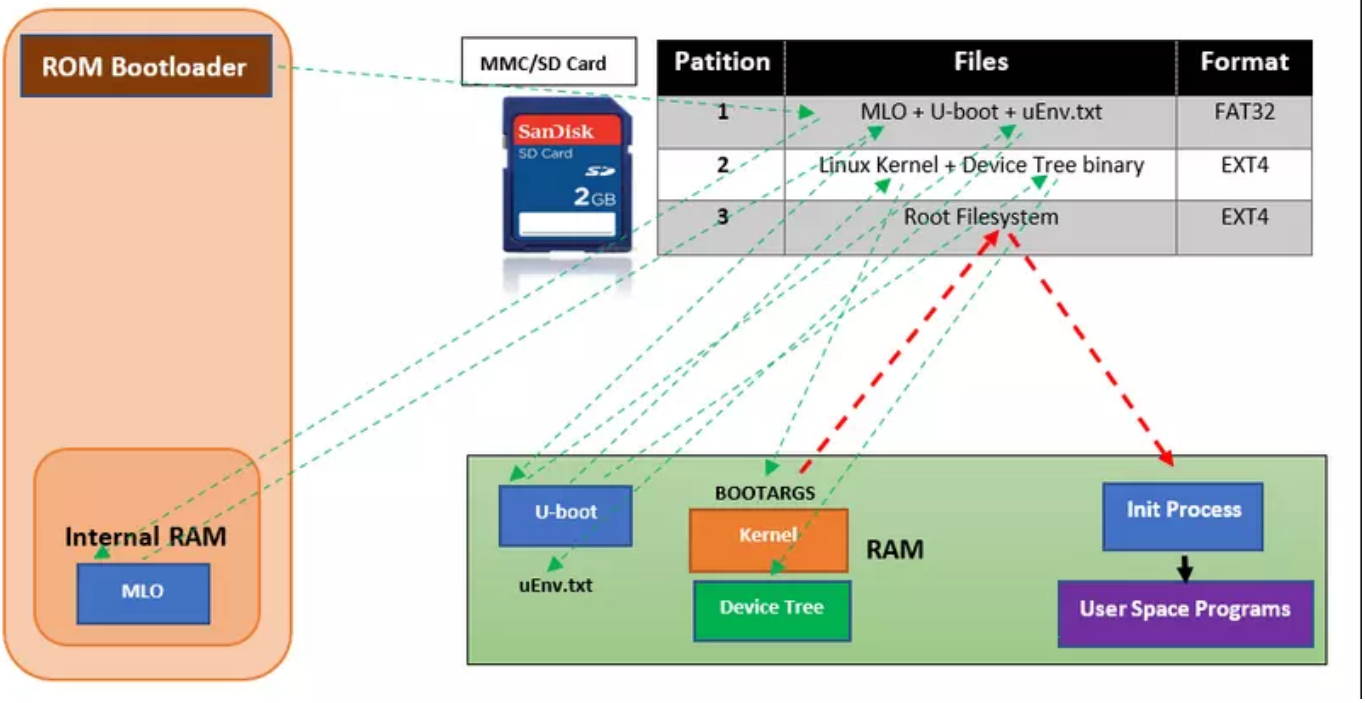


Next, U-Boot loads the kernel and device tree into RAM at addresses configured beforehand, either in the U-Boot source code or in `uEnv.txt`. Finally, it passes all kernel parameters and hands execution over to the kernel.



Linux Kernel

After receiving control and the kernel parameters from U-Boot, the kernel mounts the root filesystem (rootfs) and runs the Init process. Init is the first process started when the system boots successfully, and it runs until the system shuts down. Init starts all other child processes in user space, including the applications that interact directly with the user. At this point the system is fully ready for use.



Conclusion

The boot process of an embedded system can be divided into several stages:

- **ROM code**: Boot code written by the manufacturer that users cannot change. Its main job is to set up the system to load the SPL into internal RAM.
- **SPL**: Secondary program loader. Initializes required components and loads U-Boot into RAM.
- **U-Boot**: Loads the OS components (kernel, device tree, rootfs) into RAM, passes the kernel parameters and hands control to the kernel.
- **Linux Kernel**: Mounts the root filesystem (rootfs) and runs the Init process.

2. Kernel image

1. Fundamental concepts

- **Kernel**: a program, and the most special one in the system: it runs first, runs continuously until shutdown, and manages every other program.
- **Image**: here, a sequence of bytes packaged to be placed into memory exactly as it is in the file. A kernel image is a file containing the kernel's machine code and data, ready to be loaded into RAM and run.
- **Machine code**: binary instructions the CPU executes directly. C source code must be compiled to become machine code.

2. How a kernel image is built

- **Compilation**: each `.c` and `.S` (assembly) file is turned by the compiler into an object file `.o`, which is the machine code for that file alone.
- **Linking**: the linker combines thousands of `.o` files into a single program. It follows a linker script (a file describing how the parts of the program are laid out in memory and at which addresses). The result is `vmlinux`, an ELF file (Executable and Linkable Format, the standard executable format on Linux, containing code, data and management information such as the symbol table).
- **Format conversion**: bootloaders usually don't need the ELF information, so the `objcopy` tool strips it and keeps only the raw content, producing a raw binary (a file containing only content bytes, with no descriptive metadata).
- **Compression and packaging (architecture-dependent)**: the raw binary is compressed, and self-decompression code or a header is added, producing `zImage`, `uImage`, etc.

In summary: `.c` → `.o` → `vmlinux` (ELF) → `Image` (raw binary) → `zImage` or `uImage` (compressed and/or with header).

3. What's inside a kernel image

After linking, a program is divided into sections, each with its own properties:

Section	Contents
`.text`	Kernel machine code (scheduler, memory management, filesystems, network stack, built-in drivers)
`.rodata`	Read-only data, e.g. constant strings, `of_match_table` tables
`.data`	Initialized global variables
`.bss`	Zero-initialized global variables. The file stores only the size, not the contents; the kernel clears this area to zero at boot

Section	Contents
`.init.*`	Code and data used only during boot. After boot finishes, the kernel frees this area to reclaim RAM

The image also contains:

- **Exception vector table**: a table of addresses of the handlers the CPU invokes on interrupts or faults.
- **Built-in drivers**: drivers compiled directly into the image. When configuring the kernel (`make menuconfig`), each driver can be set to `y` (built-in, inside the image) or `m` (module, a separate `.ko` file loaded after the kernel is running).
- **Initcalls**: initialization functions of built-in drivers, called sequentially by the kernel at boot.
- **Optional**: an initramfs (a small filesystem living in RAM, used in the early stage of boot) can be embedded directly into the image.

4. Why there are multiple image formats

Format	Characteristics	Reason it exists
`vmlinux`	ELF, uncompressed, with debug info	Used for debugging (gdb), not for booting
`Image`	Raw binary, uncompressed (ARM64)	Simple: the bootloader just copies it into RAM
`zImage`	gzip-compressed + self-decompression code (ARM32)	Saves flash space
`uImage`	Adds a U-Boot header	The header holds the load address, entry point, compression type and checksum so U-Boot knows how to handle it
`bzImage`	Compressed format on x86	Similar to zImage, following the x86 boot protocol

The header in a uImage is necessary because a raw binary does not tell the bootloader which address it should be loaded to or where execution should start.

5. Role of the kernel image in the system

When the kernel image runs, the kernel is responsible for:

- **Running in privileged mode** (the level at which the CPU allows direct access to all hardware). User programs run at a restricted level. These two domains are called kernel space and user space.
- **Mediating between software and hardware**: user programs may not touch hardware directly; they call the kernel through system calls (the mechanism by which a user program asks the kernel to do something, such as `open`, `read`, `write`, `fork`).
- **Managing resources**:
  - Processes: creation, scheduling, termination.
  - Memory: RAM allocation, virtual memory (a mechanism in which each process sees its own address space), protection between processes.
  - Devices: controlling hardware through drivers.
  - Files: filesystems.
  - Networking: TCP/IP protocols, sockets.

Without a running kernel image, the hardware is just components, with no operating system to run applications.

6. The kernel image during boot

1. The bootloader (such as U-Boot) reads the kernel image from flash or an SD card into RAM, because the CPU executes quickly and directly only from RAM.
2. The bootloader also loads the DTB (hardware description) and prepares the kernel command line (a string of parameters for the kernel, e.g. `console=ttyS0 root=/dev/mmcblk0p2`).
3. The bootloader jumps to the entry point (the address of the kernel's first instruction) and passes information according to the architecture's convention. Example on ARM64: register `x0` holds the DTB address.
4. If the image is compressed, the self-decompression code runs first, decompresses the real kernel into RAM and jumps into it.
5. The kernel's first assembly code sets up a minimal environment, including enabling the MMU (Memory Management Unit, the hardware that translates virtual addresses to physical addresses). It then calls the C function `start_kernel()`.
6. `start_kernel()` initializes memory and the scheduler, parses the DTB, and runs the initcalls to initialize drivers.

7. The kernel mounts the root filesystem (where user programs and data live), then runs the first process, `/sbin/init`. From here the system switches to user space.

3. Device Tree

What is a device tree?

A device tree is a data structure describing the hardware of a board, written as text for the kernel to read, instead of having hardware information hard-coded in the kernel source.

1. The problem it solves

- **Kernel**: the core of the operating system, managing the CPU, memory and hardware.
- **Driver**: code that lets the kernel control a specific hardware device (UART, I2C, GPIO, ...).
- On the ARM architecture, hardware does not announce to the kernel which devices exist or at which addresses (unlike PCs, which have PCI/ACPI for device discovery). For example, the kernel cannot tell by itself which register address the board's UART is at or which interrupt number it uses.

Previously, this information was written directly into kernel code for each board (called board files). Every new board required modifying and compiling a separate kernel, which bloated the kernel source and made it hard to maintain.

2. The essence

The device tree separates the hardware description from the kernel code. The same kernel can run on many different boards, as long as it is given the matching hardware description file.

The device tree only describes (which devices exist, at which addresses, which interrupts, how they are connected); it contains no control logic. Control logic lives in the driver.

3. Structure

A device tree is a tree made of:

- **Node**: represents a device or a bus (a data transfer path). A node can contain child nodes.
- **Property**: a name = value pair inside a node, describing a characteristic of that device.

```
/ {
    soc {
        uart0: serial@40001000 {
            compatible = "vendor,uart-v1";
            reg = <0x40001000 0x400>;
            interrupts = <12>;
            status = "okay";
        };
    };
};
```

Common properties:

- `compatible`: a string identifying the device type, used by the kernel to pick the matching driver.
- `reg`: the start address and size of the device's register region.
- `interrupts`: the interrupt number (a signal the device uses to notify the CPU) used by the device.
- `status`: `"okay"` enables the device, `"disabled"` disables it.

4. Workflow

- **DTS (Device Tree Source)**: the human-written text file, with extension `.dts` (and `.dtsi` for shared parts).
- **DTC (Device Tree Compiler)**: the tool that compiles DTS into binary form.
- **DTB (Device Tree Blob)**: the compiled binary `.dtb` file.
- The bootloader (the first program that runs at power-on, such as U-Boot) loads the kernel and DTB into RAM, then passes the DTB address to the kernel.
- The kernel parses the DTB, creates the corresponding devices, then matches each node's `compatible` property against the compatible lists declared by drivers (usually the `of_match_table`). On a match, the driver's `probe()` function is called to initialize the device.

4. Rootfs

What is a rootfs?

Rootfs (short for root filesystem) is the first filesystem the kernel mounts, located at the root `/` of the directory tree. Every other file and directory in the system lives under it or is mounted beneath

it.

1. Fundamental concepts

- **Filesystem**: a way of organizing data on a storage device into files and directories so the kernel and programs can find, read and write them. Examples: ext4, squashfs, UBIFS.
- **Mount**: the operation by which the kernel attaches a filesystem at a point in the directory tree, called a mount point, so its contents can be accessed by path.
- **Root (`/`)**: the top-level directory of the Linux directory tree. The rootfs is the filesystem mounted at `/`.

The kernel image contains only the kernel program. The rootfs contains everything else the system needs to run: user programs, libraries, configuration files and device files.

2. Why a rootfs is needed

After booting, the kernel cannot do anything useful for the user on its own. It needs to run the first user-space program, `/sbin/init`. That program is a file, so it must live in some filesystem. The rootfs is that place.

If the kernel cannot find the rootfs, it reports a kernel panic (a fatal error where the kernel halts completely), usually with the message "VFS: Unable to mount root fs".

3. Typical rootfs contents

Directory	Contents
`/bin`, `/sbin`	Basic executables (shell, ls, init, ...)
`/lib`	Shared libraries, e.g. libc
`/etc`	System configuration files
`/dev`	Device files representing hardware, such as `/dev/ttyS0`
`/proc`, `/sys`	Virtual filesystems created by the kernel, exposing kernel and device information. Not stored on disk
`/lib/modules`	Driver modules (`.ko`)
`/usr`, `/var`, `/home`, `/tmp`	Additional programs, variable data, user directories, temporary files

4. Rootfs in embedded systems

On embedded boards, the rootfs is usually built with tools such as Buildroot or Yocto (tools that build a complete lightweight Linux system for embedded boards), and typically uses BusyBox (a single small program that bundles many basic commands).

The rootfs can live in:

- **SD card or eMMC**: an ext4 partition, as on the Raspberry Pi.
- **NAND/NOR flash**: using UBIFS or JFFS2, or squashfs (compressed, read-only).
- **RAM (initramfs)**: the rootfs is loaded directly into RAM.
- **Network (NFS)**: the rootfs lives on a server and the board mounts it over the network. Convenient during development because editing files doesn't require rewriting flash.

5. Initramfs

An initramfs is a small compressed rootfs (cpio format), loaded into RAM along with the kernel or embedded in the kernel image. The kernel unpacks it into RAM and uses it as a temporary rootfs. It is commonly used to:

- Run a small system entirely in RAM.
- Serve as an intermediate step: load the required drivers, then switch to the real rootfs on storage.

6. How the kernel finds the rootfs

The bootloader passes the kernel command line, in which the `root=` parameter specifies the rootfs location, for example:

```
root=/dev/mmcblk0p2 rootfstype=ext4 rw
```

Meaning: the rootfs is on the second partition of the SD card `mmcblk0`, formatted as ext4, mounted read-write (`rw`).

7. Where the rootfs fits in the boot process

1. The bootloader loads the kernel image and DTB into RAM.
2. The kernel runs and initializes hardware and drivers.
3. The kernel mounts the rootfs according to the `root=` parameter.
4. The kernel runs `/sbin/init` from the rootfs.
5. init starts services and user programs.

6. Init startup

1. Overview

Init is the first process the kernel runs in user space, and startup (starting services) is the work init does afterwards: bringing the system from "the kernel has just finished booting" to a ready-to-use state.

2. Role of init

- **Ancestor of all processes**: every other user-space process is spawned directly or indirectly from init, through `fork()` (the system call that creates a child process by duplicating the current process).
- **System startup**: runs the necessary scripts or services, such as mounting additional filesystems, configuring the network and starting applications.
- **Adopting orphan processes**: when a parent process exits before its child, the child becomes an orphan and is adopted by init.
- **Reaping zombie processes**: a child that has exited but whose parent has not yet read its exit status becomes a zombie (a leftover entry in the process table). Init calls `wait()` to clean them up.
- **Shutdown**: on shutdown or reboot, init stops services in order and then notifies the kernel.

If init exits or cannot run, the kernel reports a kernel panic ("Attempted to kill init!" or "No working init found").

3. The startup process

After init starts, startup usually proceeds as follows:

1. Mount the remaining filesystems (`/proc`, `/sys`, `/dev`, `/tmp`, ...).
2. Set the hostname and load required driver modules.

3. Configure the network.
4. Start background services (daemons, programs that run continuously in the background, e.g. an SSH server).
5. Start the main application or allow login via a shell (a program that accepts commands from the user).

Revision #2
Created 2026-10-23 08:57:10 UTC by trungnguyen
Updated 2026-10-23 09:03:55 UTC by trungnguyen